



APRENDIZAJE DE MÁQUINA

CALIBRACIÓN DE MODELOS

Especialización en Inteligencia Artificial
FIUBA

Dr. Ing. Facundo Adrián Lucianna
Esp. Lic. María Carina Roldán

En la clase anterior vimos ...

Aprendizaje No Supervisado. Clustering. K-Means. Hierarchical clustering. Modelo de Mixtura Gaussiana. Suma de cuadrados intracluster. Índice de la silueta. Fuerza de predicción.

APRENDIZAJE NO SUPERVISADO

¿Para que usaríamos este tipo de aprendizaje?

Lo que buscamos en aprendizaje no supervisado es descubrir **una estructura sobre la base del conjunto de datos**. A diferencia de los problemas de aprendizaje supervisado, que buscamos predecir un resultado.

En un caso particular, es cuando queremos encontrar en los datos conjuntos que responden de una forma similar, es decir agrupamientos que a priori no son evidentes.

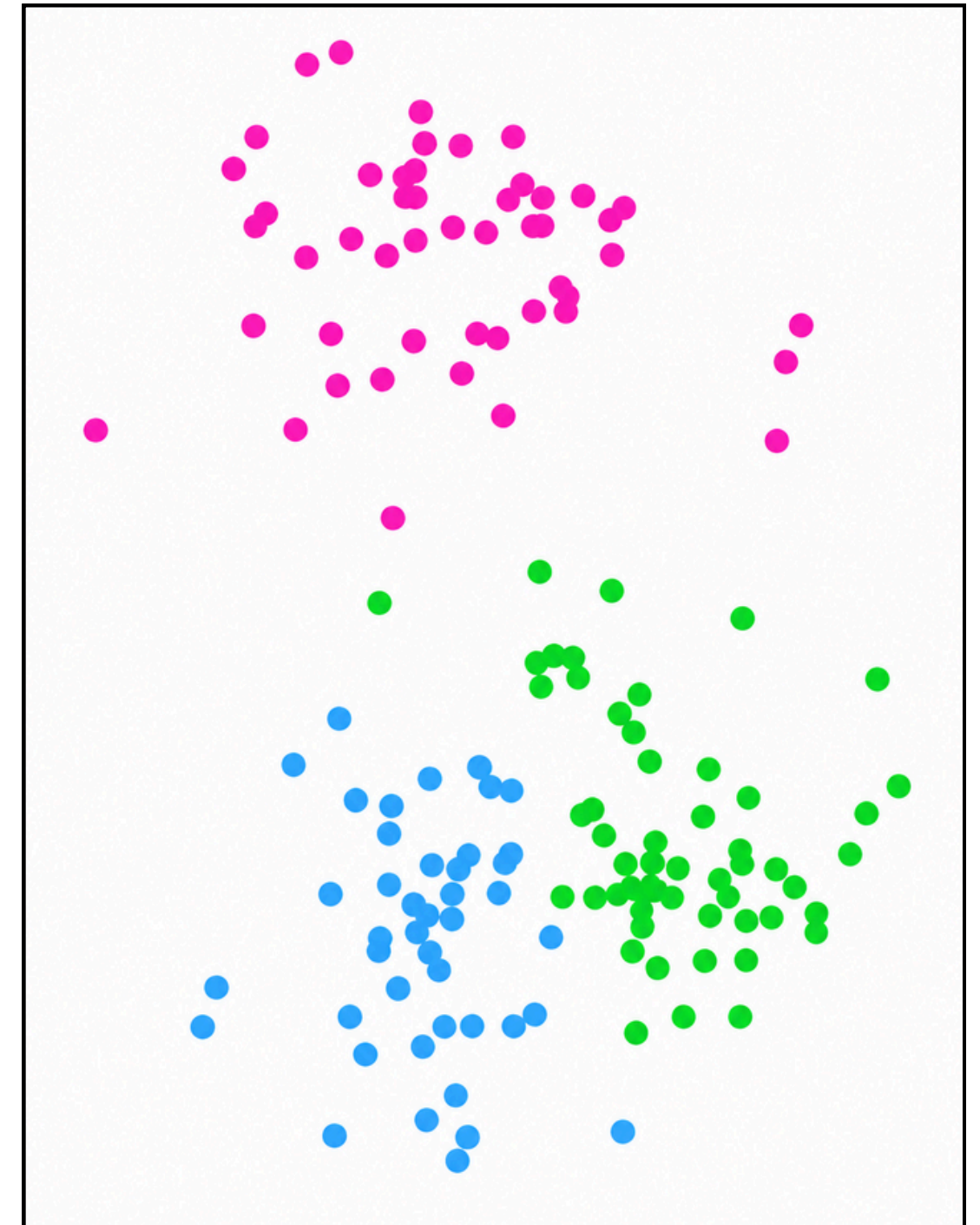
Por ejemplo, un sitio de compras podría intentar identificar grupos de compradores con historiales de navegación y compras similares, así como artículos que sean de particular interés para los compradores dentro de cada grupo.

CLUSTERING

Clustering o agrupación es un conjunto de técnicas de aprendizaje no supervisado cuyo objetivo es identificar grupos naturales dentro de un conjunto de datos.

La idea es formar grupos de manera que:

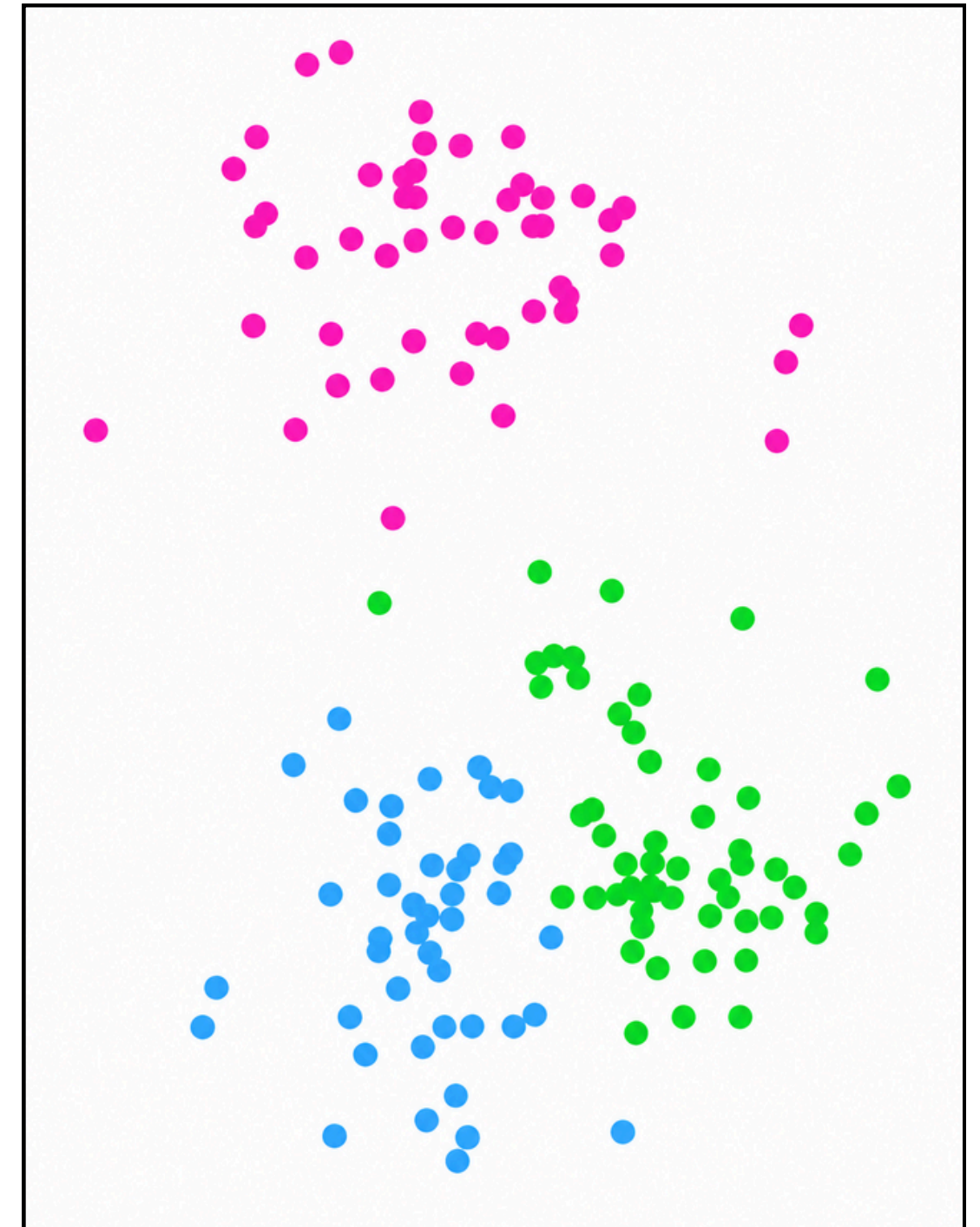
- las observaciones de un mismo grupo sean lo más **similares posible entre sí**.
- las observaciones sean muy **diferentes** de las que pertenecen a otros grupos.



CLUSTERING

Vimos tres algoritmos de clustering de los más famosos:

- **K-Means**
- **Hierarchical Clustering**
- **Modelo de Mixtura Gaussiana**

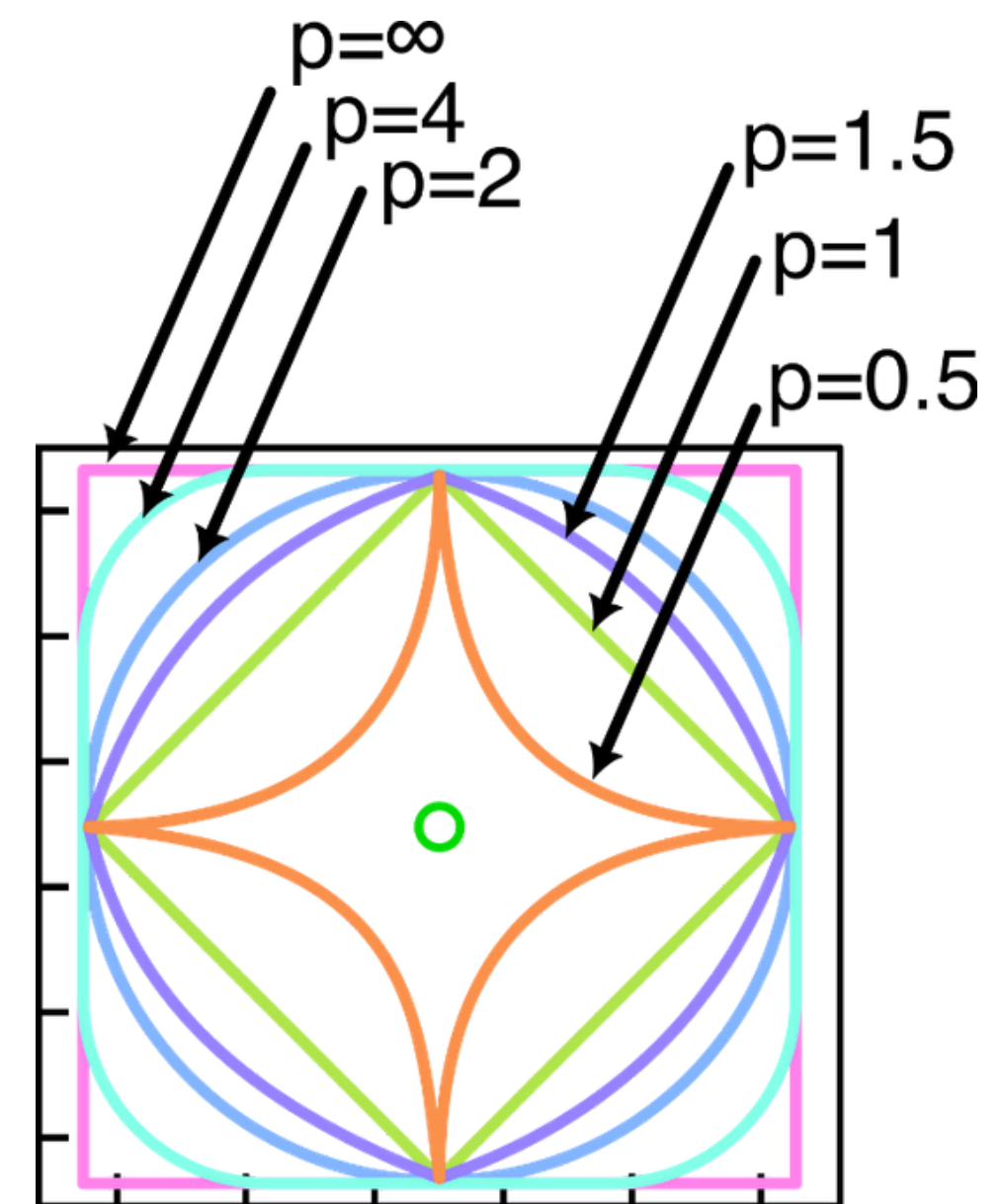


MEDIDAS DE DISTANCIA O SIMILITUD

En clustering, la similitud entre observaciones suele medirse mediante una **función de distancia**.

En muchos casos se utilizan las mismas medidas de distancia que vimos al estudiar kNN:

- Euclidiana (módulo 2)
- Manhattan, taxi, city-block (módulo 1)
- Chebyshev (módulo infinito)
- Distancia coseno
- etc.



K-MEANS

La idea detrás de K-Means es encontrar un agrupamiento en el que las observaciones de un mismo cluster sean lo más similares posible entre ellas. Para ello, se busca **minimizar la variación dentro de los clusters**.

El objetivo es minimizar:
$$\sum_{k=1}^K W(C_k)$$

donde $C_1, \dots, C_K$ representan los clusters.
y $W(C_k)$ es la variación dentro del cluster k .

K-MEANS

Una forma de resolver esto de una forma más eficiente es calcular el centroide del cluster y calcular la distancia con respecto a ese punto. El centroide de un cluster se calcula de la siguiente forma:

$$\textit{Centroide} = \frac{1}{|C_K|} \sum_{i \in C_k} x_i$$

Entonces, ahora podemos calcular la distancia intra-cluster más sencillamente:

$$W(C_k) = \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - x_j)^2$$

K-MEANS

Y ahora buscamos minimizar la suma de las distancias insta-cluster que es lo que se conoce como **inercia**:

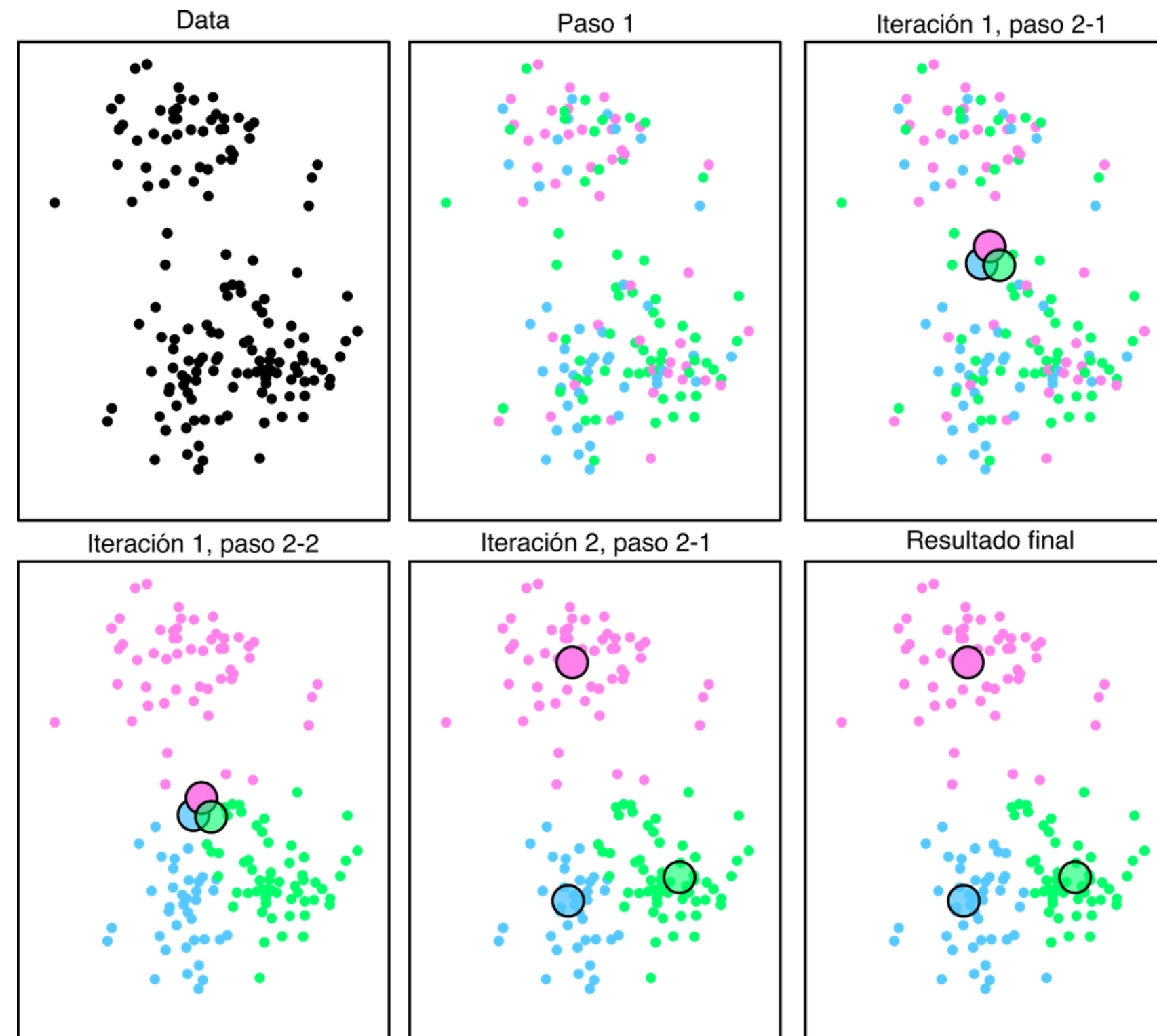
$$Inercia = \sum_{k=1}^K W(C_k)$$

Y ahora sí, el problema de minimización es más eficiente.

$$W(C_k) = \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - x_j)^2$$

El problema es iterativo, y se va minimizando de a pasos hasta que llegue a un valor estable, siempre en dirección que reduzca la **inercia**.

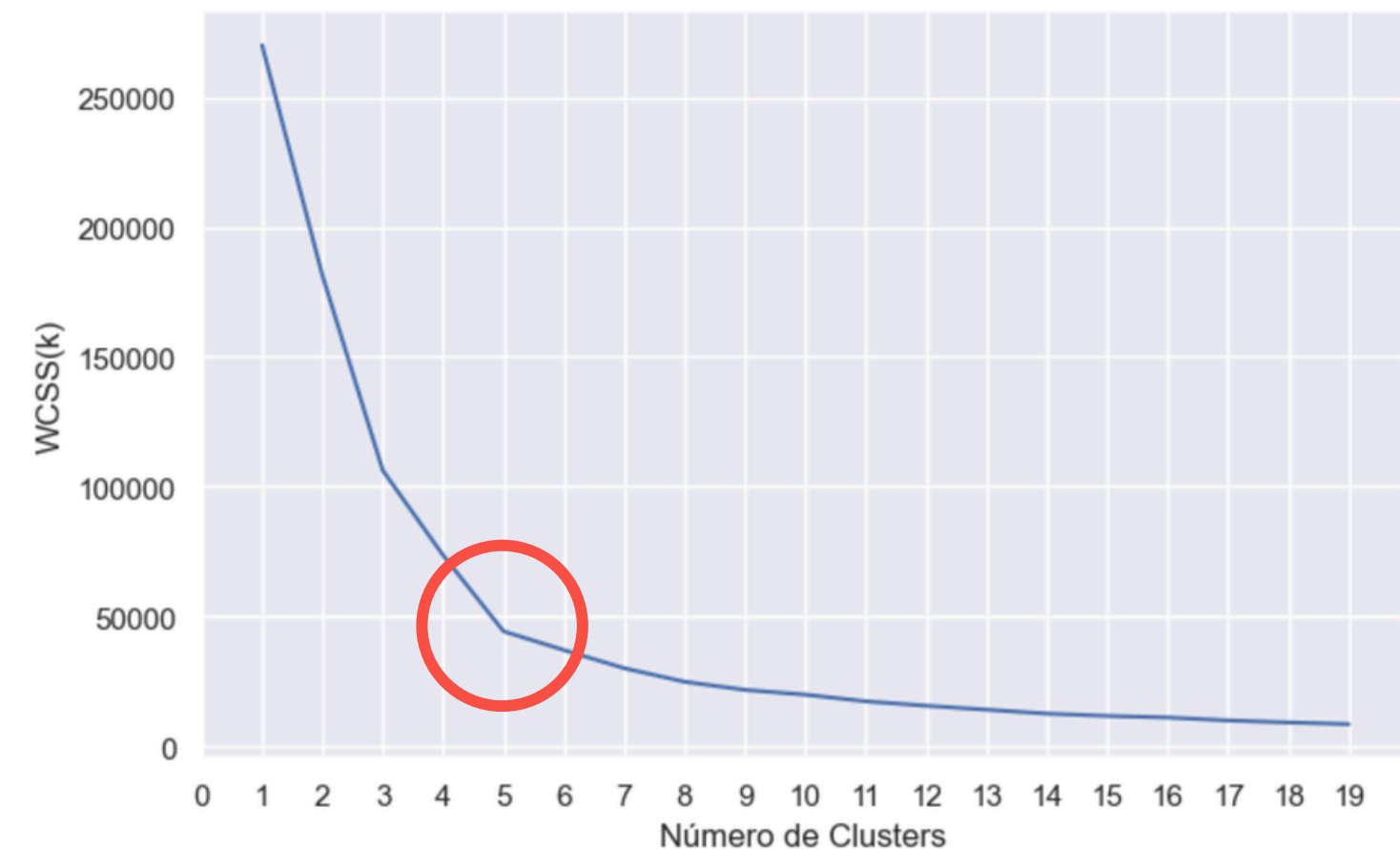
K-MEANS



MÉTODO DEL CODO (*Elbow method*)

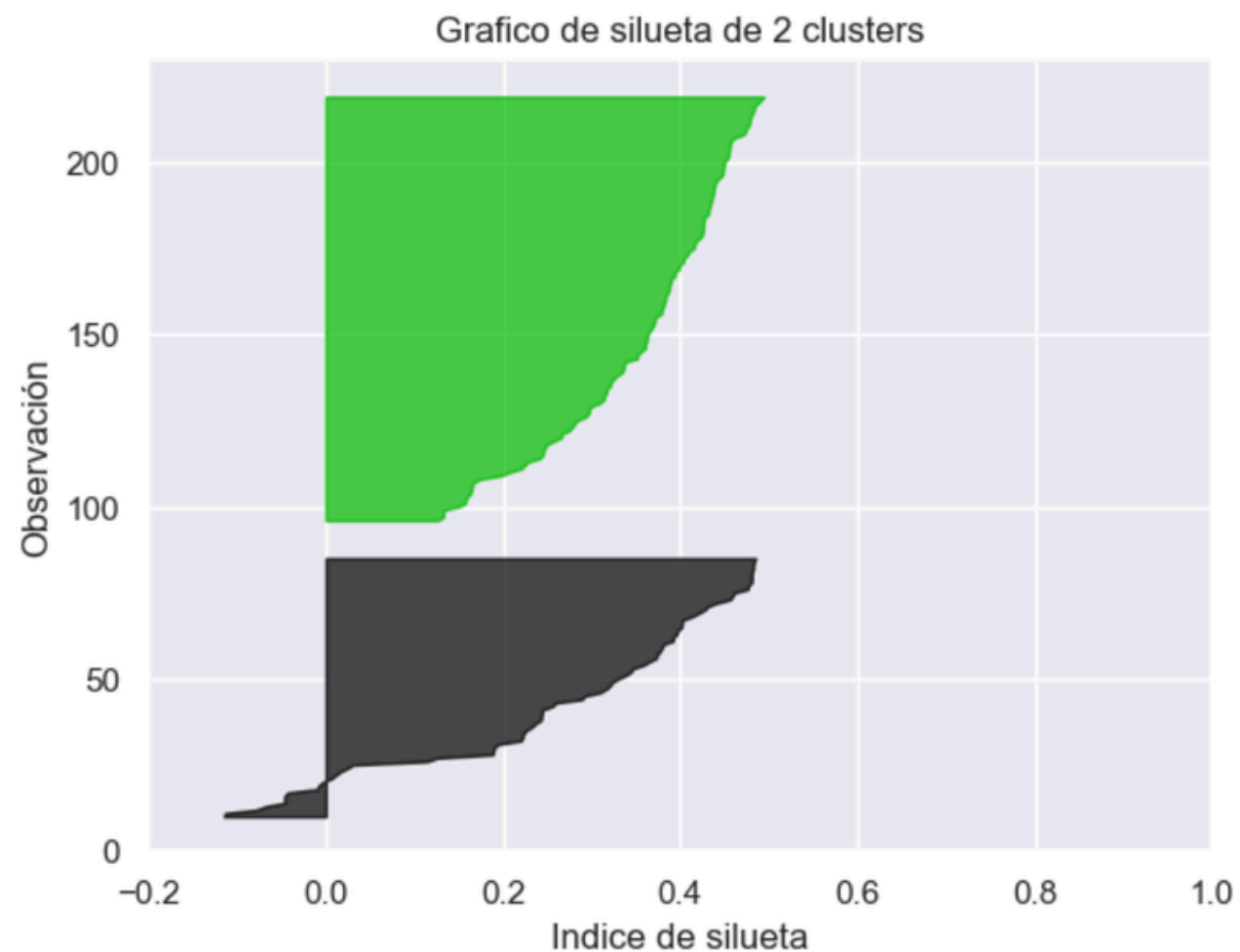
Consiste en analizar cómo varía una métrica de calidad del agrupamiento (generalmente la inercia) a medida que aumenta el número de clusters.

1. Ejecutar el algoritmo de clustering para distintos valores de K .
2. Calcular una métrica de calidad (generalmente la inercia) para cada valor de K .
3. Graficar la métrica en función de K .
4. Observar el gráfico y buscar un punto de codo. El punto de codo es aquel donde la métrica (inercia) comienza a disminuir a una tasa más lenta. En otras palabras, es el punto donde aumentar el número de clusters produce una mejora cada vez menor.



Índice de la silueta (*Silhouette Score*)

Este índice nos permite graficar los clusters (incluso cuando se tienen más de dos dimensiones), ver el tamaño de cada uno y evaluar qué tan segura es la pertenencia de cada observación a su cluster.

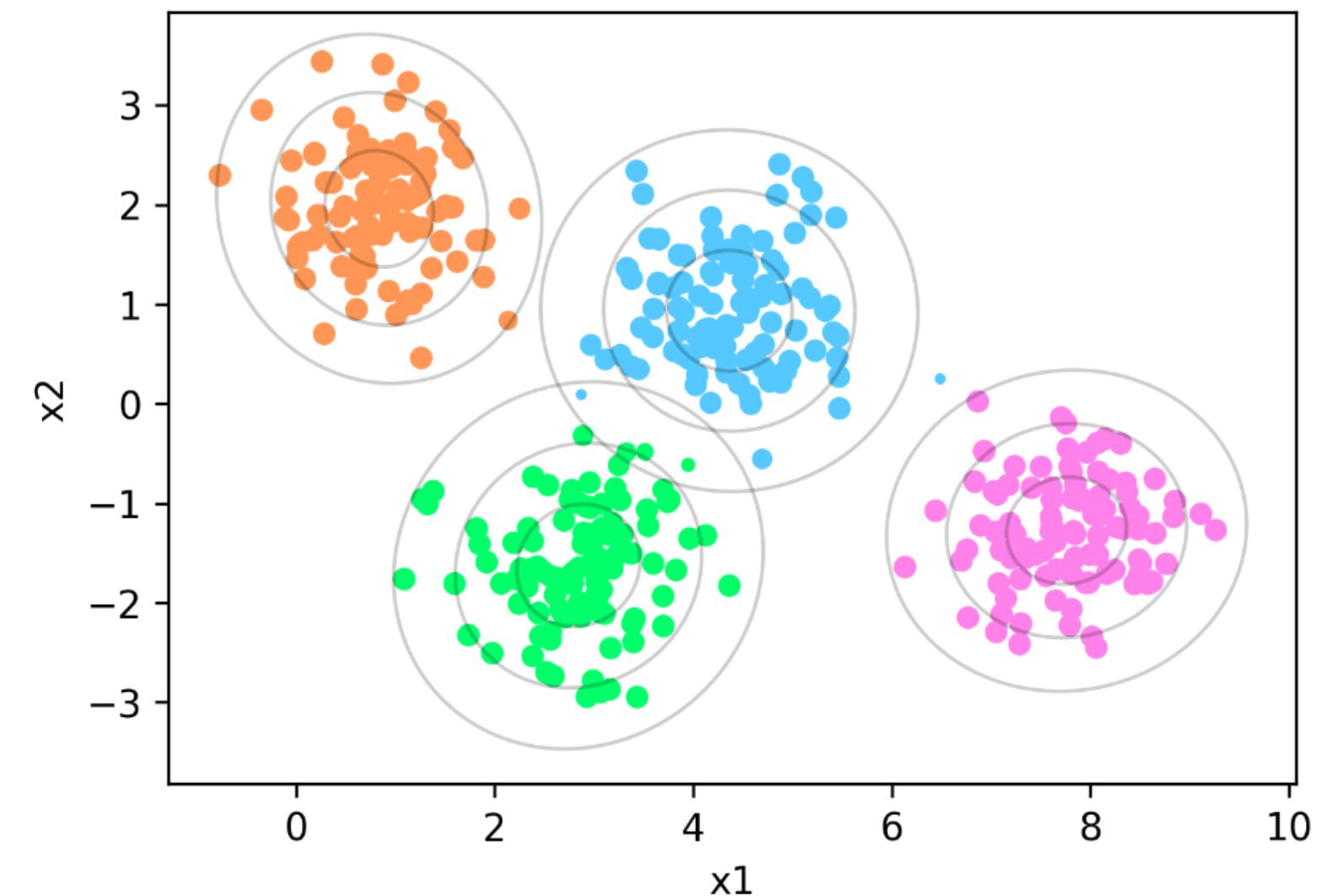


MODELOS DE MIXTURA GAUSSIANA

Un **Gaussian Mixture Model (GMM)** intenta encontrar una mezcla de distribuciones de probabilidad gaussianas multidimensionales que modelen mejor cualquier conjunto de datos de entrada.

En el caso más simple, los GMM se pueden utilizar para encontrar conglomerados de la misma manera que k-means.

Pero debido a que GMM contiene un modelo probabilístico, también es posible encontrar asignaciones probabilísticas.

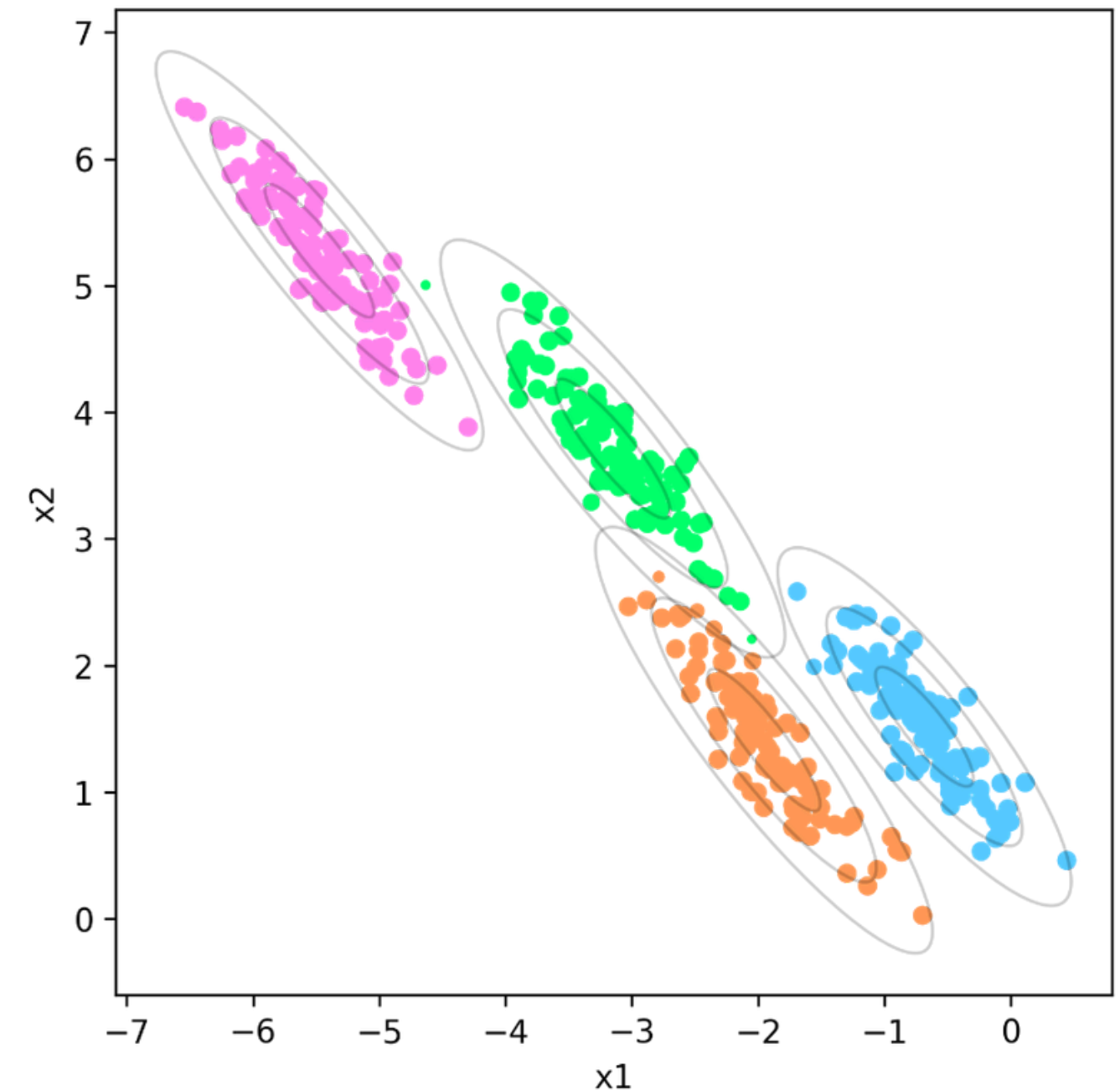


MODELOS DE MIXTURA GAUSSIANA

Un **Gaussian Mixture Model (GMM)** intenta encontrar una mezcla de distribuciones de probabilidad gaussianas multidimensionales que modelen mejor cualquier conjunto de datos de entrada.

En el caso más simple, los GMM se pueden utilizar para encontrar conglomerados de la misma manera que k-means.

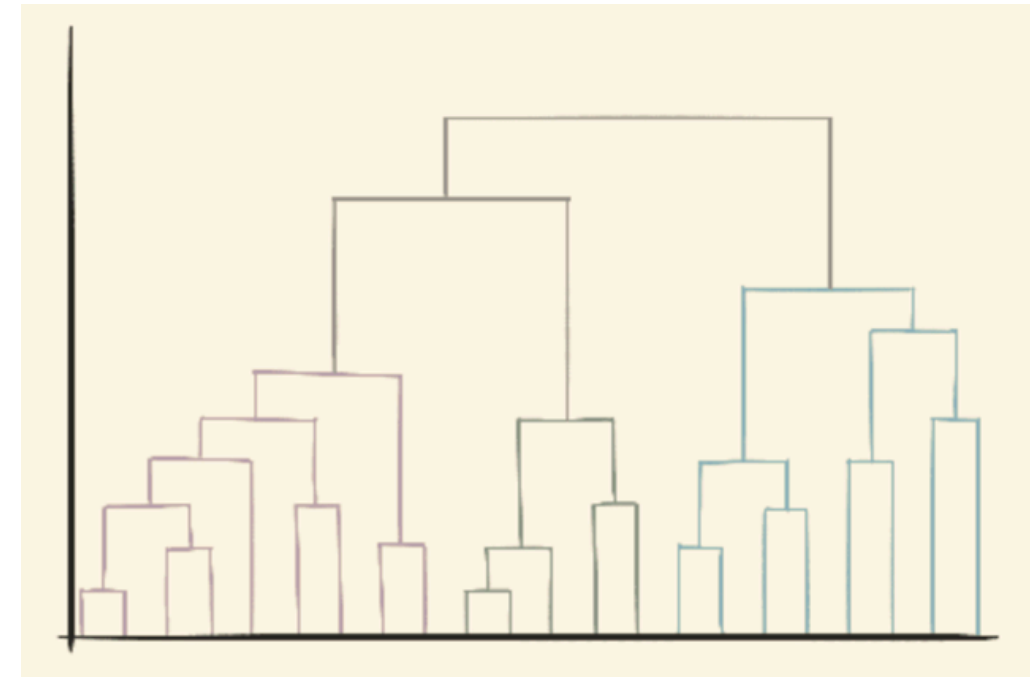
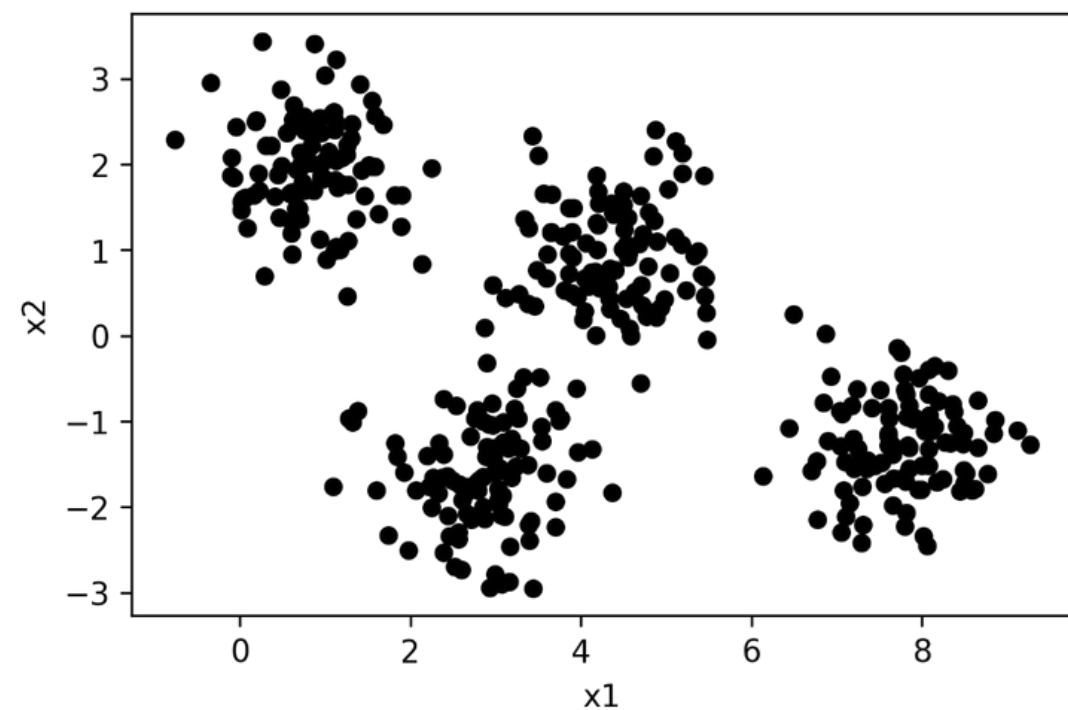
Pero debido a que GMM contiene un modelo probabilístico, también es posible encontrar asignaciones probabilísticas.



HIERARCHICAL CLUSTERING

Una (potencial) desventaja de los modelos anteriores es que requieren que especifiquemos el número de clusters.

El **clustering jerárquico** es una alternativa que no tiene ese prerequisite.



HIERARCHICAL CLUSTERING

El **clustering jerárquico** es una técnica de aprendizaje no supervisado cuyo objetivo es organizar las observaciones en una **estructura jerárquica** según su similitud.

A diferencia de otros métodos de clustering, no produce una única partición de los datos, sino una secuencia de agrupamientos representada mediante una estructura en forma de árbol.

Una de sus principales ventajas es que permite explorar diferentes números de clusters sin necesidad de especificarlos previamente, lo que facilita la interpretación y el descubrimiento de patrones en los datos.

HIERARCHICAL CLUSTERING. Dendrograms.

Un **dendrograma** es una representación visual de los datos y la jerarquía de clusters a la que pertenecen. Cada hoja es una observación.

Gráficamente, se pueden identificar diferentes números de clusters trazando una línea horizontal y deslizándola hacia arriba o hacia abajo.

La altura del corte determina el número de clusters (# de líneas verticales atravesadas por el corte).



CALIBRACIÓN DE MODELOS

CALIBRACIÓN DE MODELOS

Hasta ahora, cada vez que evaluamos un clasificador nos preguntamos lo mismo: **¿acierta?**

Medimos accuracy, armamos la matriz de confusión, miramos el AUC–ROC.

Hoy vamos a hacernos una pregunta distinta, que ninguna de esas métricas responde: cuando el modelo devuelve **0.7**, ¿ese número se corresponde con algo real del mundo?

La respuesta no es obvia. En muchos modelos, la respuesta es "**casi nada**": el número está en el intervalo $[0,1]$, tiene aspecto de probabilidad, pero no fue construido para serlo.

Y si ese número se usa para decidir —otorgar un crédito, priorizar un paciente, frenar un auto— el problema deja de ser estadístico y pasa a ser costoso.

CALIBRACIÓN DE MODELOS

Todas las métricas de clasificación que vimos en las clases anteriores responden a la misma pregunta:

Matriz de confusión	Cuántos aciertos y errores de cada tipo comete el modelo
Accuracy	Proporción global de aciertos
Precision / Recall / F1	Calidad de la clase predicha, según qué error nos preocupe
Sensibilidad / Especificidad	Desempeño sobre cada clase por separado
AUC-ROC	Calidad del ordenamiento , barriendo todos los umbrales posibles

Todas responden "**¿acierta?**". Ninguna responde "**¿el número que devuelve es creíble?**". Hoy incorporamos un eje de evaluación nuevo, que es **independiente** de todos estos.

CALIBRACIÓN DE MODELOS

Repaso: el AUC solo mira el orden, no los valores

Tomemos un modelo A y construyamos un modelo B elevando al cuadrado todos sus scores:

Obs	Modelo A	Modelo B	y (real)
1	0.20	0.04	0
2	0.40	0.16	0
3	0.60	0.36	1
4	0.80	0.64	1
	AUC = 0.95	AUC = 0.95	

El ranking de las instancias es **idéntico** en los dos modelos, así que el AUC no cambia ni un decimal. Pero los números son completamente distintos: donde A decía "*0.80 de probabilidad*", B dice "*0.64*".

Esto vale para **cualquier transformación monótona** del score. Si el AUC es ciego a los valores, necesitamos otra herramienta que sí los mire.

CALIBRACIÓN DE MODELOS

¿Qué es la calibración?

Trabajemos sobre clasificación binaria. Decimos que un modelo está **calibrado** si sus probabilidades predichas coinciden con las frecuencias realmente observadas:

De todos los casos a los que el modelo asignó una probabilidad de 0.7, el evento efectivamente ocurre en el 70% de las veces.

Formalmente, si el modelo produce un score $\mathbf{s}(\mathbf{X})$ entre $[0,1]$:

$$P(y = 1 | s(X) = p) = p \quad \text{para todo } p \text{ entre } [0,1]$$

Dos detalles de la definición que importan:

- La igualdad debe valer **en todo el rango de scores**, no solamente en promedio. Un modelo puede acertar la tasa base global y estar mal calibrado en cada tramo por separado.
- La condición se plantea sobre **grupos** de instancias con el mismo score, no sobre una instancia individual. La calibración es una propiedad de frecuencias, y por eso solo se puede verificar sobre un conjunto de datos.

CALIBRACIÓN DE MODELOS

Lo que la definición NO dice

Prestemos atención a todo lo que la definición de calibración deja afuera:

- **No dice que el modelo acierte.** Un modelo calibrado puede equivocarse en la mayoría de sus predicciones.
- **No dice que separe bien las clases.** La calibración no exige ninguna capacidad discriminativa.
- **No dice nada sobre el ranking.** No pide que los positivos reciban scores más altos que los negativos.
- **Solo dice que el número es interpretable como una frecuencia.**

La calibración y la capacidad predictiva son propiedades separadas, y hay que medirlas por separado. Un modelo puede ser excelente en una y pésimo en la otra, en cualquiera de las dos direcciones.

CALIBRACIÓN DE MODELOS

Un contraejemplo

Supongamos un dataset donde el **30% de los mails son spam**. Proponemos el siguiente modelo:

Responder 0.30 para todos los mails, sin leer el contenido.

¿Está calibrado? Sí, y de forma perfecta. De todos los casos a los que asignó 0.30, exactamente el 30% resultan ser spam. La definición se cumple con igualdad exacta.

¿Sirve para algo? No. No distingue un mail de otro: le asigna el mismo score a un mail legítimo de trabajo y a uno con "GRATIS" en mayúsculas. Su **AUC es 0.5**, el de un modelo aleatorio.

Este ejemplo deja las dos ideas separadas de forma limpia: **calibración perfecta, utilidad nula**. Calibrar no es una forma de mejorar el modelo, es una forma distinta de evaluarlo.

DISCRIMINACIÓN vs CALIBRACIÓN

Discriminación es la capacidad de **ordenar**: ¿el modelo le asigna scores más altos a los positivos que a los negativos? Es lo que miden todas las métricas que ya conocemos. Dependen del ranking o de un umbral, nunca del valor absoluto del score.

Calibración es la capacidad de **cuantificar**: ¿los números coinciden con las frecuencias reales?

Son propiedades independientes, y las cuatro combinaciones existen:

	Bien calibrado	Mal calibrado
Discrimina bien	El ideal: ordena bien y los números significan algo	 Acá cae casi todo lo que usamos: Random Forest, SVM, Naive Bayes, boosting, redes profundas
Discrimina mal	El modelo constante de la diapositiva anterior	El peor caso posible

El cuadrante que nos ocupa el resto de la clase es el de **arriba a la derecha**.

DISCRIMINACIÓN vs CALIBRACIÓN

El cuadrante peligroso, con números

Veamos concretamente cómo se ve un modelo del cuadrante superior derecho.

Un modelo asigna 0.99 a casi todos los casos que resultan positivos y 0.01 a los negativos:

- **Ordena perfecto** → AUC $\approx$ 1.0, accuracy altísimo, precision y recall excelentes
- Por cualquier métrica de clasificación, es un modelo sobresaliente
- **Pero** de todas las instancias a las que les puso 0.99, solo el **40%** resultan efectivamente positivas

Como **clasificador** es excelente: si lo único que queremos es decidir la clase, funciona perfecto.

Como **estimador de probabilidad** miente de forma sistemática: dice "99% de certeza" donde la realidad es 40%.

Y si alguien usa ese 0.99 para calcular un riesgo, el error no es el 1% que sugiere el número: es del 59%.

DISCRIMINACIÓN vs CALIBRACIÓN

Cuándo la calibración NO importa

Antes de convencerlos del problema, conviene ser honestos sobre sus límites.

Mientras la salida del modelo se use **solo para elegir una clase con un umbral fijo**, la calibración es prácticamente irrelevante:

- La decisión es simplemente "*¿el score supera el umbral?*"
- Cualquier transformación monótona del score da **la misma clase predicha**
- El ranking se preserva, así que ninguna métrica de clasificación cambia

Si tu sistema clasifica con umbral fijo y nadie mira las probabilidades, podés ignorar todo lo que sigue.

El problema aparece en el momento, muy frecuente, en que alguien empieza a usar el número.

DISCRIMINACIÓN vs CALIBRACIÓN

Cuándo la calibración SI importa

La calibración se vuelve crítica cuando **la probabilidad entra en el cálculo**, y no solamente la clase:

- **Valor esperado**: cualquier decisión de la forma $p \times \text{ganancia} + (1 - p)\text{perdida}$ usa p directamente como número, no como orden.
- **Matriz de costos**: cuando el umbral óptimo se deriva de los costos de cada tipo de error, ese cálculo supone que las probabilidades son reales.
- **Priorización**: ordenar una cola de casos por riesgo y decidir dónde cortar según capacidad operativa.
- **Umbrales distintos de 0.5**: todo umbral calculado a partir de las probabilidades hereda su error.
- **Comunicar el resultado a una persona**: un médico o un analista que recibe "riesgo del 15%" va a interpretarlo literalmente.

En todos estos casos, un número mal calibrado propaga el error directamente a la decisión

DISCRIMINACIÓN vs CALIBRACIÓN

Caso: otorgamiento de préstamos

Una entidad financiera usa un modelo para estimar la probabilidad de incumplimiento (*default*) de cada solicitante.

Lo que dice el modelo:

- Para el cliente X, la probabilidad estimada de default es **$p = 0.10$**
- Interpretación natural: de 100 clientes con ese mismo score, esperamos que unos **10** no paguen

Lo que pasó a fin de mes:

- De esos 100 clientes, 22 no pagaron

El modelo estaba subestimando el riesgo real a menos de la mitad de su valor. No porque ordenara mal, los clientes más riesgosos efectivamente recibían scores más altos, sino porque la **escala** de sus números no se correspondía con ninguna frecuencia observable.

DISCRIMINACIÓN vs CALIBRACIÓN

Caso: otorgamiento de préstamos

Ajustamos el modelo contra las frecuencias observadas. El mismo cliente X pasa de 0.10 a **p = 0.22**.

Ahora comparemos las dos situaciones usando el umbral estándar de 0.5:

	Modelo sin calibrar	Modelo calibrado
Probabilidad de default	0.10	0.22
¿Supera el umbral de 0.5?	No	No
Clase predicha	No default	No default
Acurracy, precision, recall	--	Sin cambios

La clase predicha es **la misma**. Ninguna métrica de clasificación se movió. **¿Entonces da igual?**

DISCRIMINACIÓN vs CALIBRACIÓN

Caso: otorgamiento de préstamos

Nadie decide con la clase predicha: se decide con el **valor esperado**.

Préstamo de \$1000. Ganancia por intereses si el cliente paga: \$200. Pérdida del capital si no paga.

Decisión con el modelo sin calibrar ($p = 0.10$):

$$E[\text{resultado}] = 0.10 \times (-\$1000) + 0.90 \times (+\$200) = -\$100 + \$180 = +\$80$$

El préstamo parece rentable → **se otorga**.

Decisión con el modelo calibrado ($p = 0.22$):

$$E[\text{resultado}] = 0.22 \times (-\$1000) + 0.78 \times (+\$200) = -\$220 + \$156 = -\$64$$

El préstamo destruye valor → **no se otorga**.

La clase predicha era idéntica en los dos casos. **La decisión de negocio es opuesta**. Y como esta decisión se repite miles de veces por mes, la diferencia no es de \$144: es de \$144 por operación, sostenida en el tiempo.

DISCRIMINACIÓN vs CALIBRACIÓN

Dónde aparece esto en la práctica

El patrón se repite en todo dominio donde la diferencia entre 51% y 99% cambia el curso de acción:

-  **Medicina.** Decidir si se interviene, se deriva o se espera. Un riesgo del 10% y uno del 40% llevan a protocolos distintos, aunque **ninguno de los dos supere el umbral de "positivo"**.
-  **Finanzas.** Scoring crediticio, pricing de seguros, provisiones por riesgo. La probabilidad es literalmente un insumo contable.
-  **Vehículos autónomos.** Un sistema que estima 0.6 de que haya un peatón debe comportarse distinto que si estima 0.99, aunque en ambos casos "detecte peatón".
-  **Sistemas de IA en dominios sensibles.** La confianza reportada por el modelo se usa para decidir cuándo escalar a un humano. **Un modelo con exceso de confianza nunca escala.**
-  **Selección de umbrales.** El umbral "óptimo" derivado de una matriz de costos está mal ubicado si las probabilidades están mal.

FAMILIA DE MODELOS

FAMILIA DE MODELOS

Probabilidades ingenuas

Si la calibración fuera automática, no estaríamos hablando de esto. Los modelos producen lo que llamamos **probabilidades ingenuas**:

*Números en $[0,1]$ que **parecen** probabilidades pero que nunca fueron optimizados para serlo.*

Son un subproducto del algoritmo, no su objetivo. El modelo optimiza otra cosa, el margen, la impureza de los nodos, el error residual, y la "probabilidad" sale como un efecto lateral de esa optimización.

El sesgo más común es el **exceso de confianza**: predicciones empujadas hacia 0 y 1 sin justificación. Pero no es el único, y hay modelos con el problema exactamente inverso.

Vale la pena entender **por qué falla cada familia**, porque la causa determina el remedio.

FAMILIA DE MODELOS

Naive Bayes: exceso de confianza extremo

Síntoma. Casi todas las predicciones se acumulan cerca de 0.001 o de 0.999. Ver un 0.6 es raro, ver un 0.9999 es lo habitual.

Causa. El modelo asume que todas las *features* son **condicionalmente independientes** dada la clase. Cuando esa hipótesis no se cumple, y casi nunca se cumple, el modelo multiplica verosimilitudes de evidencia que en realidad está correlacionada, y termina contando la misma señal varias veces.

Ejemplo. Un mail contiene "GRATIS", "OFERTA" y "URGENTE".

- Esas tres palabras casi siempre aparecen juntas en el mismo tipo de mail: son **una sola** señal.
- Naive Bayes las trata como **tres** piezas de evidencia independientes y multiplica sus verosimilitudes.
- La probabilidad resultante se dispara hacia el extremo.

Cuantas más features correlacionadas tenga el problema, más pronunciado es el efecto.

FAMILIA DE MODELOS

SVM: la salida directamente no es una probabilidad

Síntoma. No hay ninguna probabilidad que corregir. El método *decision_function()* devuelve números como -3.7 o +12.4: sin cota superior ni inferior, sin escala interpretable.

Causa. El SVM optimiza el **margen geométrico**: busca el hiperplano que separa las clases dejando la mayor distancia posible a los puntos más cercanos. Su salida natural es **la distancia con signo al hiperplano**.

Una distancia no es una probabilidad:

- No está acotada en $[0,1]$
- Nada garantiza que un punto al doble de distancia sea "el doble de seguro"
- La escala depende de las unidades de las features y del valor de C

Este es el caso más claro de por qué la calibración no es opcional sino **constitutiva**: sin ella, un SVM simplemente **no puede responder** a la pregunta "*¿con qué probabilidad?*".

FAMILIA DE MODELOS

Random Forest: demasiado conservador

Síntoma. El problema exactamente inverso al de Naive Bayes. Las probabilidades se concentran en el centro del rango y rara vez se acercan a 0 o a 1.

Causa. La probabilidad se estima como la **fracción de árboles que votan** por la clase. Para que el ensamble emita un 1.0, los 500 árboles deben coincidir **sin una sola excepción**.

Y eso casi nunca ocurre, por construcción:

- Cada árbol se entrena sobre una muestra *bootstrap* distinta
- Cada división considera solo un subconjunto aleatorio de variables
- Esa diversidad forzada es lo que hace que el ensamble generalice bien

En otras palabras: **la varianza entre árboles es lo que hace bueno al Random Forest, y es exactamente lo que le impide ser tajante**. El mecanismo que le da su calidad predictiva es el que le arruina la calibración.

FAMILIA DE MODELOS

Gradient Boosting (XGBoost, LightGBM)

Cada árbol nuevo se entrena para corregir el error residual de los anteriores, lo que empuja sistemáticamente las predicciones de los casos ya bien clasificados hacia el extremo correcto. Con suficientes iteraciones y sin regularización adecuada, el modelo termina **sobreajustando su propia confianza** sobre el conjunto de entrenamiento: predicciones polarizadas en 0.01 y 0.99

Redes neuronales profundas

La combinación de sobreparametrización y entrenamiento prolongado con *cross-entropy* lleva a la red a seguir reduciendo la pérdida mucho después de haber alcanzado el mínimo error de clasificación. Y una vez que todos los ejemplos están bien clasificados, **la única forma de seguir bajando la pérdida es aumentar la confianza**.

Es un fenómeno bien documentado y algo incómodo: las arquitecturas modernas **discriminan mejor** que las de hace una década y están **peor calibradas**.

FAMILIA DE MODELOS

Regresión logística: el caso especial

Es el único modelo de la lista que optimiza directamente la probabilidad.

Al minimizar *log loss*, la regresión logística está maximizando la verosimilitud del modelo de Bernoulli: busca los parámetros que hacen que las probabilidades predichas se parezcan lo más posible a las frecuencias observadas.

La calibración no es un accidente afortunado: es la función de costo

⚠ Pero la garantía vale solo bajo condiciones, y esas condiciones se violan seguido:

- **Regularización suave.** Una penalización *L1* o *L2* agresiva encoge los coeficientes y comprime las probabilidades hacia la tasa base. Con un *C* chico en scikit-learn, la calibración se degrada.
- **Forma funcional correcta.** Si la relación real entre las variables y el *log-odds* no es lineal, el modelo está mal especificado y no hay verosimilitud que lo salve.
- **Sin cambio de distribución.** La garantía vale sobre la distribución de entrenamiento, bajo *dataset shift* se pierde.

Es un buen punto de partida, no una garantía. **Siempre verificar empíricamente**, incluso en los modelos que "deberían" estar calibrados.

FAMILIA DE MODELOS

El factor que no depende del algoritmo

Hay una causa de descalibración que no depende del modelo y que en la práctica es **la más frecuente de todas**: haber alterado la proporción de clases durante el entrenamiento.

La secuencia típica:

1. El dataset real tiene un 2% de positivos (fraude, enfermedad rara, churn)
2. Aplicamos *undersampling*, *oversampling*, *SMOTE* o *class_weight='balanced'* para llevarlo a 50%
3. El modelo aprende a estimar probabilidades sobre la **distribución remuestreada**, no sobre la real
4. Resultado: el modelo predice **0.5** donde la frecuencia verdadera es **0.02**

Es una sobreestimación sistemática y masiva de la clase minoritaria.

Esto no es un defecto del método de balanceo. Esas técnicas mejoran la discriminación y por eso las usamos. Es un efecto colateral esperable, y la respuesta correcta es **calibrar después, sobre un conjunto con la proporción real de clases.**

MEDICIÓN DE LA CALIBRACIÓN

MEDICIÓN DE LA CALIBRACIÓN

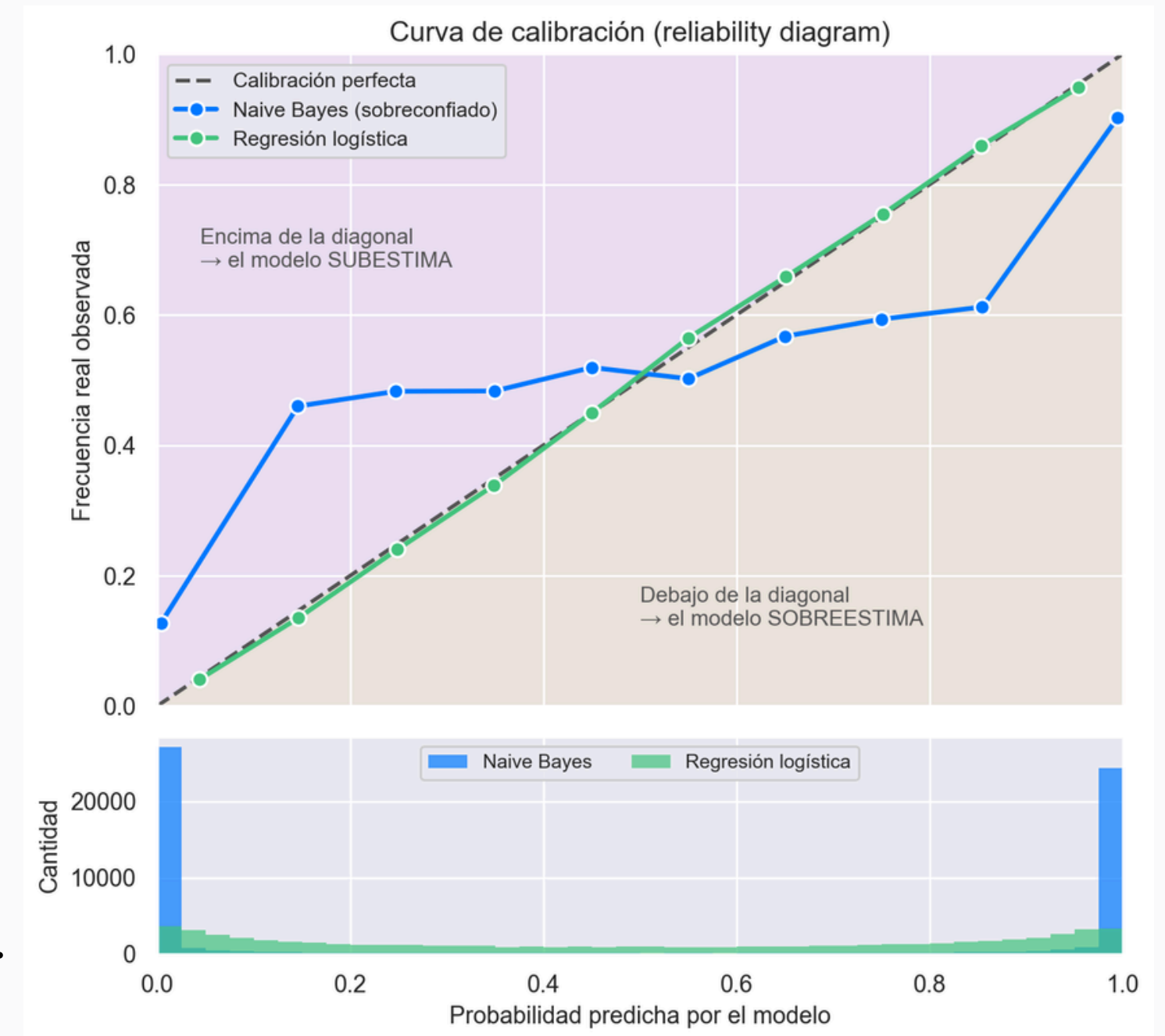
Reliability Diagram o Curva de calibración

No se puede corregir lo que no se mide. La primera herramienta es visual: un gráfico que compara, para cada nivel de probabilidad predicha, la frecuencia real observada.

Construcción, paso a paso:

1. **Binning**. Se ordenan las predicciones y se dividen en M intervalos, típicamente M=10: [0.0,0.1), [0.1,0.2), ...
2. **Eje X**. Para cada bin, el promedio de las probabilidades predichas
3. **Eje Y**. Para cada bin, la proporción real de positivos observados
4. Se grafican los M puntos y se traza la diagonal $y = x$ como referencia

La diagonal representa la calibración perfecta: predicho = observado. Todo apartamiento de esa recta es error de calibración.



MEDICIÓN DE LA CALIBRACIÓN

Reliability Diagram o Curva de calibración

Ubicación de la curva	Qué significa
Sobre la diagonal $y=x$	Calibración perfecta
Por debajo de la diagonal	La frecuencia real es menor que la predicha → el modelo sobreestima (exceso de confianza)
Por encima de la diagonal	La frecuencia real es mayor que la predicha → el modelo subestima
Forma de "S" acostada	Patrón típico de sobreconfianza: exagera en ambos extremos a la vez

El truco para no confundirse: el eje Y es la realidad. Si la curva está **abajo**, la realidad quedó por debajo de lo que el modelo prometía.

La forma de la curva también informa qué método de corrección conviene: una "S" limpia sugiere Platt Scaling, una curva irregular, regresión isotónica.

MEDICIÓN DE LA CALIBRACIÓN

Reliability Diagram o Curva de calibración

Es un detalle que se pasa por alto y que cambia la lectura por completo:

- Un punto **lejísimos** de la diagonal, correspondiente a un bin con **4 observaciones** → es puro ruido, no dice nada
- Una desviación **chica** en un bin con **5000 observaciones** → es un problema real y sistemático
- En la curva los dos se ven exactamente igual. Sin el histograma de las probabilidades predichas, no hay forma de distinguirlos.

 **Además, la curva depende del binning:**

- **Pocos bins** → se promedian regiones distintas y la curva parece mejor de lo que realmente es
- **Muchos bins** → cada uno queda con pocas muestras y la curva se vuelve ruidosa e ilegible
- No existe una elección universalmente correcta: conviene **probar dos o tres valores de M** y quedarse con las conclusiones que se sostienen en todos.

MEDICIÓN DE LA CALIBRACIÓN

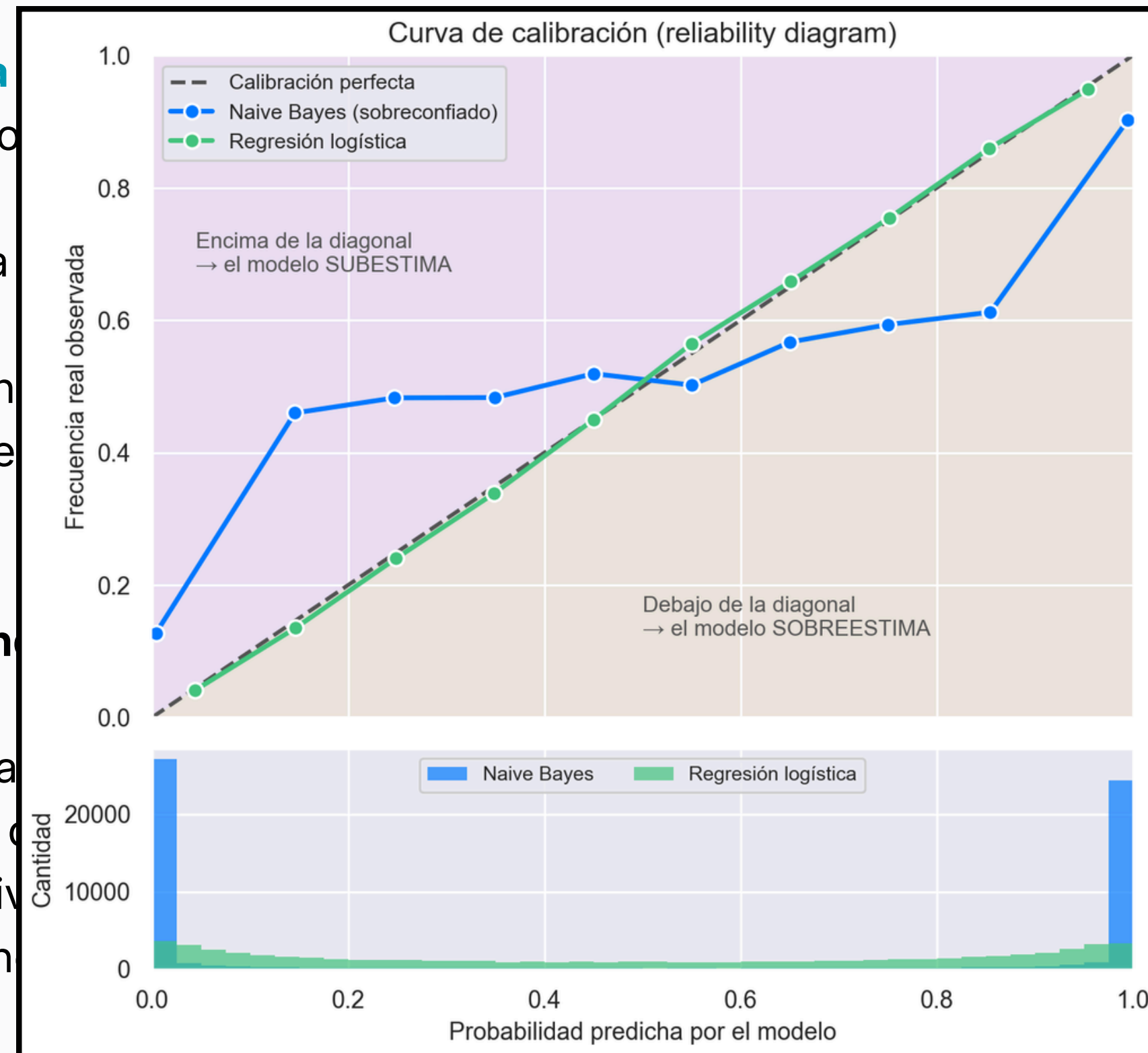
Reliability Diagram o Curva

Es un detalle que se pasa por

- Un punto **lejísimos** de la nada
- Una desviación **chica** en
- En la curva los dos se ven de distinguirlos.

⚠ **Además, la curva depende**

- **Pocos bins** → se promedian
- **Muchos bins** → cada uno
- No existe una elección unívoca de conclusiones que se sostienen



→ es puro ruido, no dice

sistemático
es predichas, no hay forma

mente es
ilegible
de **M** y quedarse con las

MEDICIÓN DE LA CALIBRACIÓN

Brier Score

La métrica cuantitativa más usada. Es, simplemente, el error cuadrático medio entre la probabilidad predicha y el resultado observado:

$$BrierScore = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

donde N es el número de observaciones, p_i la probabilidad predicha para la instancia i , e y_i la etiqueta real.

- **Rango [0,1], menor es mejor.** El 0 corresponde a predicciones perfectas y perfectamente confiadas.
- Es una regla de **puntuación propia**: el Brier esperado se minimiza exactamente cuando el modelo reporta la probabilidad que realmente cree. Exagerar hacia 0 o 1 empeora el puntaje. Si se compara con el accuracy, al que le da lo mismo un 0.7 honesto que un 1.0 inventado: por eso no sirve para evaluar calibración.

Comparación con log loss. Tiene la misma propiedad, con una diferencia práctica importante: log loss penaliza los errores confiados mucho más severamente (un $p = 0.001$ para un caso positivo tiende a infinito). Eso lo hace más sensible pero también más inestable.

MEDICIÓN DE LA CALIBRACIÓN

Brier Score

El Brier Score no es una métrica pura de calibración. Se puede descomponer en tres términos:

$$\text{Brier Score} = \text{Calibración} + \text{Ruido} - \text{Discriminación}$$

Agrupando en K bins con n_k observaciones, probabilidad media $\bar{p}_k$ y frecuencia observada $\bar{y}_k$, siendo $\bar{y}$ la tasa base:

- **Calibración:** Confiabilidad $= \frac{1}{N} \sum_k n_k (\bar{p}_k - \bar{y}_k)^2$ queremos que sea **chica**
- **Discriminación:** Resolución $= \frac{1}{N} \sum_k n_k (\bar{y}_k - \bar{y})^2$ queremos que sea **grande**
- **Ruido:** Incertidumbre $= \bar{y}(1 - \bar{y})$ no depende del modelo.

⚠ **No comparar el Brier entre modelos distintos** si su capacidad de discriminación difiere: no se sabe cuál de los términos explica la diferencia.

✅ **Sí es válido comparar el mismo modelo antes y después de calibrar:** como calibrar no altera el ranking, la resolución queda fija y **toda la mejora observada es calibración**. Ese es su uso natural en esta clase.

MEDICIÓN DE LA CALIBRACIÓN

Expected Calibration Error (ECE)

El Brier mide calibración y discriminación juntas. A veces queremos un número que mida **solo** el error de calibración: tomar la curva y resumir en un escalar cuánto se aparta de la diagonal.

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} \left| \bar{y}_m - \bar{p}_m \right|$$

donde $|B_m|$ es la cantidad de observaciones en el bin m .

Es el promedio de las distancias verticales a la diagonal, **ponderado por cuántas observaciones caen en cada bin**, es decir, exactamente la corrección que hacíamos "a ojo" mirando el histograma, pero convertida en número.

Su interpretación es directa: ECE = 0.08 significa que, en promedio, las probabilidades del modelo se desvían **8 puntos porcentuales** de la frecuencia real

MEDICIÓN DE LA CALIBRACIÓN

Variante pesimista — Maximum Calibration Error

$$\text{MCE} = \max_m \left| \bar{y}_m - \bar{p}_m \right|$$

Reporta el **peor bin** en lugar del promedio. Es la métrica relevante en aplicaciones críticas, donde no alcanza con estar bien calibrado *en promedio*.

⚠ ECE al depender del binning hereda sus problemas, y **no** es una regla de puntuación propia: el modelo que siempre da el mismo valor tiene $\text{ECE} \approx 0$ y no sirve para nada. Por eso el ECE **complementa** al Brier, no lo reemplaza.

MEDICIÓN DE LA CALIBRACIÓN

Las tres herramientas anteriores comparten un requisito que no se puede relajar: **se calculan sobre datos que el modelo no vio durante el entrenamiento.**

Evaluar la calibración sobre el conjunto de entrenamiento es **aún más engañoso** que evaluar el accuracy ahí, y la razón es contraintuitiva:

*Los modelos sobreajustados están, sobre sus propios datos de entrenamiento, **casi perfectamente calibrados**. Memorizaron las etiquetas, así que sus scores coinciden con las frecuencias observadas por construcción.*

Un Random Forest sin podar produce una curva de calibración **casi perfecta** sobre *train* y una **desastrosa** sobre *test*. Si medimos sobre train, concluiríamos que no hay nada que corregir.

MÉTODOS DE CALIBRACIÓN

MÉTODOS DE CALIBRACIÓN

¿Cómo corregimos un modelo mal calibrado?

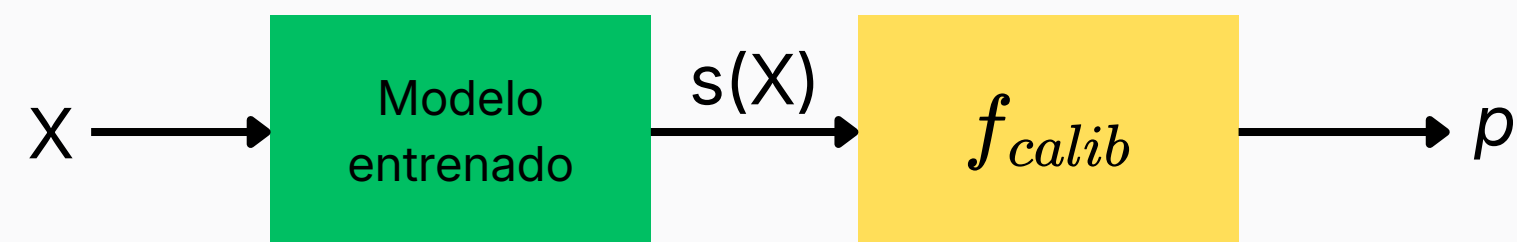
La estrategia para corregir es simple. **No se modifica el modelo original**. Se aprende una función adicional que transforma su salida.

Dado un modelo entrenado que produce scores $s(X)$, buscamos f_{calib} tal que:

$$f_{calib}(s(X)) \approx P(y = 1 | X)$$

Es un **post-procesamiento**, y eso tiene tres consecuencias prácticas importantes:

- **No hace falta reentrenar**. Se puede calibrar un modelo ya productivo, uno costoso de entrenar, o incluso uno del que solo tenemos las predicciones guardadas.
- **Funciona sobre cualquier score, sea o no una probabilidad**. Por eso se puede calibrar un SVM: f_{calib} toma la distancia al hiperplano y devuelve una probabilidad. El score solo tiene que ordenar razonablemente bien.
- f_{calib} **es monótona creciente** → no altera el orden de las instancias.



MÉTODOS DE CALIBRACIÓN

¿Cómo corregimos un modelo mal calibrado?

La afirmación "calibrar no cambia el desempeño" se repite mucho y conviene precisarla, porque tiene dos excepciones.

Sobre el AUC:

- **Platt Scaling** es *estrictamente* creciente → el AUC queda **exactamente** igual
- **Regresión isotónica** es *no decreciente* → distintos scores pueden colapsar al mismo valor calibrado, y esos empates pueden mover el AUC levemente (en general de forma despreciable)

Sobre el accuracy: sí puede cambiar.

Aunque el orden se preserve, la calibración **mueve los scores respecto del umbral**. Si un caso valía 0.55 y queda en 0.45, cruzó el umbral de 0.5 y su clase predicha cambió.

*Se preserva el **ranking**, no las **decisiones a umbral fijo***

En la práctica el efecto suele ser chico, pero no es cero, y hay que verificarlo después de calibrar.

MÉTODOS DE CALIBRACIÓN

Nunca calibrar con los datos de entrenamiento del modelo base

Es la fuente de error más común, y vale la pena entender exactamente **por qué falla**:

1. Sobre sus propios datos de entrenamiento, el modelo base tiene un desempeño **artificialmente bueno**: sus scores están sesgados hacia las etiquetas que memorizó.
2. Si ajustamos f_{calib} sobre esos scores optimistas, la función aprende a corregir una descalibración que no es **la que el modelo tiene sobre datos nuevos**.
3. En producción, esa corrección se aplica a scores que tienen una distorsión distinta, y va en la **dirección equivocada**.

Resultado: el modelo calibrado así funciona peor que el modelo sin calibrar

No es un error que degrade un poco el resultado: es un error que **empeora activamente** el modelo.

MÉTODOS DE CALIBRACIÓN

Opción A — Conjunto de calibración separado (*hold-out*)

Se parte el dataset en tres:

[----- train 60% -----]	[--- calibración 20% ---]	[--- test 20% ---]
Entrena el modelo	entrena f_{calib}	evalúa todo

Simple, barato y transparente. Su costo es doble: se sacrifican datos que el modelo podría haber usado para entrenar, y f_{calib} se ajusta sobre una única partición, por lo que puede ser inestable si ese conjunto es chico.

MÉTODOS DE CALIBRACIÓN

Opción B — Validación cruzada

1. Para cada fold i : se entrena el modelo base sobre los otros $k-1$ folds y se predice sobre el fold i
2. Como cada fold fue predicho por un modelo que **no lo vio**, se obtienen scores fuera de muestra para **todas** las observaciones
3. Se ajusta f_{calib} sobre ese conjunto completo

Se aprovechan todos los datos y el calibrador es mucho más estable. El costo es computacional: k entrenamientos del modelo base.

En ambos casos: el conjunto de test queda afuera de todo esto. Si la calibración lo toca, las métricas finales dejan de ser una estimación honesta.

MÉTODOS DE CALIBRACIÓN

Los métodos de calibración más utilizados son:

- Platt scaling
- Regresión isotónica

MÉTODOS DE CALIBRACIÓN

Platt scaling

El método paramétrico clásico. La idea: ajustar una **regresión logística auxiliar** que toma como única variable de entrada el score del modelo original.

$$P(y = 1 \mid X) = \frac{1}{1 + \exp(A \cdot s(X) + B)}$$

Los parámetros A y B se aprenden por máxima verosimilitud sobre el conjunto de calibración. Solo **dos parámetros** para todo el ajuste, lo que lo hace muy difícil de sobreajustar.

Qué hace cada parámetro:

- **A, la pendiente.** Controla cuán abrupta es la transición de 0 a 1. Un $|A|$ chico **aplana** la curva y reduce la confianza, que es exactamente lo que necesita un modelo sobreconfiado.
- **B, el desplazamiento.** Corre la curva horizontalmente, corrigiendo un sesgo sistemático en una dirección.

Supuesto central: la relación entre el score y la probabilidad real tiene **forma sigmoidea**. Cuando se cumple, con muy pocos datos alcanza; cuando no, la sigmoide no puede representar la distorsión.

Nota sobre la fórmula: está escrita con $+A \cdot s(X)$ en el exponente, en lugar del menos habitual de la sigmoide. Por eso A resulta típicamente negativo. Es la convención del paper original de Platt (1999), que es también la que usa scikit-learn.

MÉTODOS DE CALIBRACIÓN

Cuándo usar Platt Scaling

- **Modelos cuyo score no es una probabilidad, en especial SVM.** Es el caso para el que Platt diseñó el método originalmente.
- **Conjuntos de calibración chicos** (menos de ~1000 observaciones). Con solo dos parámetros, el riesgo de sobreajuste es mínimo.
- **Cuando la curva de calibración observada tiene la forma de "S"** característica de la sobreconfianza, que es justamente la forma que una sigmoide puede corregir.

MÉTODOS DE CALIBRACIÓN

Regresión isotónica

La alternativa no paramétrica. **No asume ninguna forma funcional**. La única restricción que impone es que la función sea **monótona no decreciente**:

$$\hat{y}_i \leq \hat{y}_j \quad \text{siempre que} \quad s(x_i) \leq s(x_j)$$

Formalmente, resuelve un problema de mínimos cuadrados con restricción de orden:

$$\min_{\hat{y}} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{sujeto a} \quad \hat{y}_1 \leq \hat{y}_2 \leq \dots \leq \hat{y}_n$$

El resultado es una función escalonada que se adapta a la forma real de los datos:

- **Tramos constantes** donde la frecuencia empírica es similar
- **Salto** donde la frecuencia cambia

Al no imponer una forma, puede representar distorsiones que una sigmoide jamás capturaría: curvas con codos, mesetas, o comportamientos distintos en distintos tramos del rango.

MÉTODOS DE CALIBRACIÓN

Regresión isotónica

El problema se resuelve con **PAVA** (*Pool Adjacent Violators Algorithm*), cuya lógica es de una simplicidad notable:

1. Recorrer la secuencia **ordenada por score**.
2. Cada vez que un valor sea **menor** que el anterior (una violación de la monotonicidad), **fusionar** los dos grupos y reemplazar ambos por su **promedio**.
3. Repetir hasta que no queden violaciones.

Un detalle importante del algoritmo: al fusionar dos grupos, el nuevo promedio puede quedar por debajo del grupo que está más atrás, generando una violación nueva. Por eso el proceso **se propaga hacia atrás** hasta estabilizarse.

Es un algoritmo de complejidad lineal y converge siempre a la solución óptima del problema de mínimos cuadrados con restricción de orden.

MÉTODOS DE CALIBRACIÓN

Regresión isotónica - PAVA: traza paso a paso

Seis observaciones **ordenadas por score**, con sus etiquetas reales:

	A	B	C	D	E	F
s(X)	0.10	0.20	0.40	0.60	0.80	0.90
y	0	0	1	0	1	1

MÉTODOS DE CALIBRACIÓN

Regresión isotónica - PAVA: traza paso a paso

Seis observaciones **ordenadas por score**, con sus etiquetas reales:

	A	B	C	D	E	F
s(X)	0.10	0.20	0.40	0.60	0.80	0.90
y	0	0	1	0	1	1

Paso 1 - Buscar violaciones. Recorremos comparando cada valor con el anterior:

- A a B: $0 \leq 0$ ✓
- B a C: $0 \leq 1$ ✓
- C a D: $1 \leq 0$ ✗ ← **violación**

*D tiene un score **mayor** que C ($0.60 > 0.40$) pero una etiqueta **menor** ($0 < 1$). No podemos asignarle una probabilidad menor si su score es mayor.*

MÉTODOS DE CALIBRACIÓN

Regresión isotónica - PAVA: traza paso a paso

Seis observaciones **ordenadas por score**, con sus etiquetas reales:

	A	B	C	D	E	F
s(X)	0.10	0.20	0.40	0.60	0.80	0.90
y	0	0	0.5	0.5	1	1

Paso 2 — Fusionar el par en conflicto. Reemplazamos C y D por su promedio:

$$\hat{y}_C = \hat{y}_D = \frac{1 + 0}{2} = 0.5 \quad \Rightarrow \quad [0, 0, \underbrace{0.5, 0.5}_{\text{fusionados}}, 1, 1]$$

MÉTODOS DE CALIBRACIÓN

Regresión isotónica - PAVA: traza paso a paso

Seis observaciones **ordenadas por score**, con sus etiquetas reales:

	A	B	C	D	E	F
s(X)	0.10	0.20	0.40	0.60	0.80	0.90
y	0	0	0.5	0.5	1	1

Paso 3 — Verificar la propagación. Reemplazamos C y D por su promedio:

- B a C,D: $0 \leq 0.5$ ✓
- C,D a E: $0.5 \leq 1$ ✓

*No hay más violaciones. **El algoritmo termina.***

MÉTODOS DE CALIBRACIÓN

Regresión isotónica - Resultado final del ajuste

Obs	s(X)	y real	Prob. Calib.
A	0.10	0	0.0
B	0.20	0	0.0
C	0.40	1	0.5
D	0.60	0	0.5
E	0.80	1	1.0
F	0.90	1	1.0

La función aprendida es escalonada: 0.0 para scores bajos, un escalón intermedio de 0.5 en la zona ambigua, y 1.0 para scores altos. Para calibrar una observación nueva se ubica su score en este mapa, interpolando entre escalones.

⚠ **Miremos el resultado con atención:** con 6 observaciones, la función tiene apenas **3 niveles**, y dos de ellos son 0.0 y 1.0 **exactos**, una confianza absoluta que ningún dato justifica.

Esto no es un defecto del ejemplo: es el defecto del método. La regresión isotónica se ajusta tan de cerca a los datos que, con pocas observaciones, sobreajusta descaradamente. De ahí su requisito de datos abundantes.

MÉTODOS DE CALIBRACIÓN

Comparando a ambos métodos

Característica	Platt Scaling	Regresión isotónica
Tipo	Paramétrico	No paramétrico
Forma de la función	Sigmoide suave	Escalonada
Parámetros	2 (A y B)	Tantos como escalones
Supuesto	Distorsión sigmoide	Solo monotonidad
Datos necesarios	Pocos	Muchos
Riesgo de sobreajuste	Bajo	Alto con pocos datos
Flexibilidad	Limitada	Alta

MÉTODOS DE CALIBRACIÓN

Comparando a ambos métodos

Guía práctica:

1. Conjunto de calibración con **menos de ~1000 observaciones** → **Platt**, sin dudarlo. La isotónica va a sobreajustar.
2. **Más de ~5000** → probar las dos y comparar el Brier Score sobre el conjunto de test.
3. El modelo es un **SVM** → Platt es el punto de partida natural.
4. La curva muestra una distorsión **irregular**, no sigmoidea → isotónica, si hay datos suficientes.
5. **En caso de duda: probar ambas y medir**. Es barato y la respuesta es empírica, no teórica.

MÉTODOS DE CALIBRACIÓN

Calibración multiclase: One-vs-Rest

Platt Scaling y la **regresión isotónica** son **binarios por naturaleza**: ambos transforman *un* score en *una* probabilidad. Para un problema de K clases hay que extenderlos.

Paso 1 — Calibración independiente por clase. El problema se descompone en K tareas binarias ("*clase k* " contra "*todas las demás*"), cada una con su propio calibrador:

$$p_k = f_{\text{calib}}^{(k)}(s_k(X)) \quad k = 1, \dots, K$$

Paso 2 — Normalización L1. Como cada calibrador se entrenó por separado, sin ninguna restricción que los vincule, las probabilidades resultantes **no suman 1**. Se corrige dividiendo por la suma:

$$P(y = k \mid X) = \frac{p_k}{\sum_{j=1}^K p_j}$$

⚠ Limitación. Este esquema calibra bien cada probabilidad **marginal**, pero no garantiza que la distribución **conjunta** sobre las K clases sea coherente, porque la normalización es un arreglo posterior y no algo que los calibradores optimizaron. Para K chico funciona bien, para K grande existen métodos específicos (*Dirichlet calibration*, *matrix/vector scaling*), y temperature scaling (deep learning) es multiclase de forma nativa.

CONCLUSIONES

Errores frecuentes

1. **Calibrar con los datos de entrenamiento del modelo base.** El error número uno. Produce un modelo peor que no calibrar.
2. **Creer que calibrar mejora el modelo.** No lo mejora como clasificador: el AUC no cambia. Mejora la **interpretabilidad de sus probabilidades**, que es otra cosa.
3. **Suponer que un AUC alto implica buena calibración.** Son propiedades independientes: un modelo con AUC 0.95 puede tener un ECE pésimo.
4. **Confiar en la regresión logística sin verificar.** Está calibrada bajo condiciones que se violan seguido.
5. **Resamplear y no corregir la tasa base.** Si se entrenó con SMOTE o undersampling, el conjunto de calibración debe tener la **proporción real** de clases.
6. **Usar regresión isotónica con pocos datos.** Produce probabilidades de 0.0 y 1.0 exactas.
7. **Comparar el Brier Score entre modelos con discriminación muy distinta.** La métrica mezcla ambas propiedades.
8. **Leer la curva de calibración sin mirar el histograma.** Los bins con pocas observaciones producen desviaciones espectaculares que son puro ruido.
9. **No recalcular el umbral de decisión después de calibrar.** Los scores se movieron, así que el umbral óptimo cambió.



Vamos a practicar un
poco ...

Próxima materia:

Operaciones de aprendizaje automático I